
Data Structures

BS (CS) _Fall_2025

Lab_14 Manual



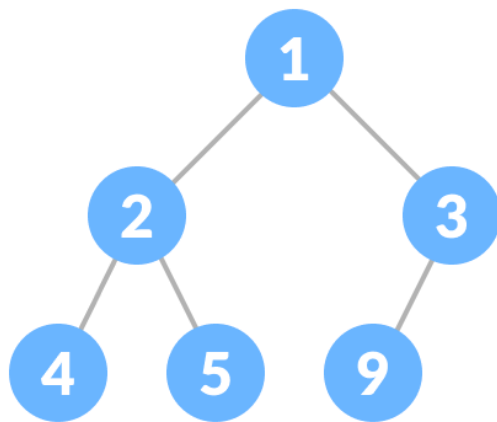
Learning Objectives:

1. Heaps and Graph

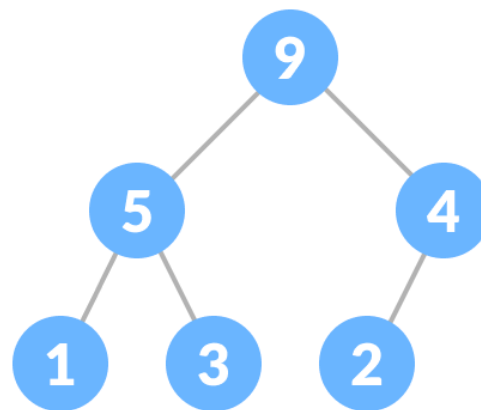
Heap Data Structure

Heap data structure is a complete binary tree that satisfies **the heap property**, where any given node is

- always greater than its child node/s and the key of the root node is the largest among all other nodes. This property is also called **max heap property**.
- always smaller than the child node/s and the key of the root node is the smallest among all other nodes. This property is also called **min heap property**.



Min Heap



Max Heap

This type of data structure is also called a **binary heap**.

Heap Operations

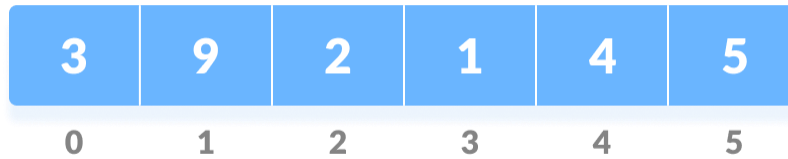
There are several essential operations used with heaps:

1. **Insert:** Adds a new element to the heap while maintaining the heap property (either min or max heap ordering).
2. **Extract Min/Max:** Removes and returns the minimum/maximum element from the heap.
3. **Peek:** Returns the minimum/maximum element without removing it.
4. **Heapify:** Reorganizes a subtree for a given node to ensure the heap property holds.
5. **Delete:** Removes a specific element from the heap.
6. **Increase/Decrease Key:** Changes the value of an existing element in the heap.
7. **Build Heap:** Converts an array into a proper min or max heap.

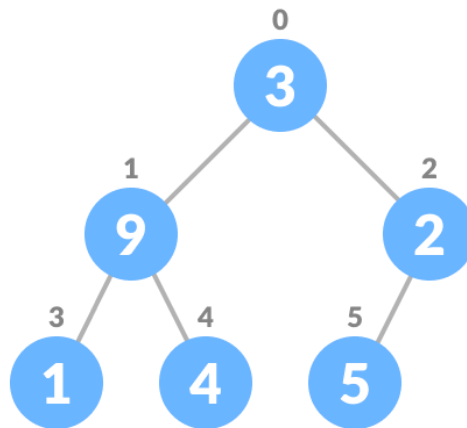
Heapify

Heapify is the process of creating a heap data structure from a binary tree. It is used to create a Min-Heap or a Max-Heap.

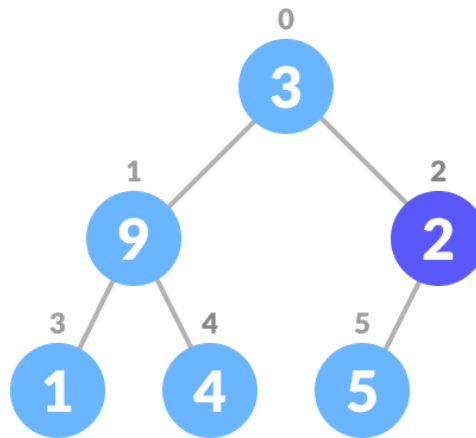
1. Let the input array be



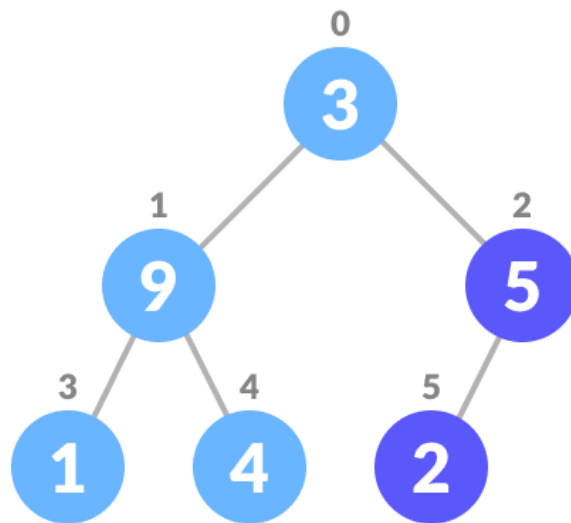
2. Create a complete binary tree from the array



3. Start from the first index of non-leaf node whose index is given by $n/2 - 1$.



4. Set current element i as largest.
5. The index of left child is given by $2i + 1$ and the right child is given by $2i + 2$.
 - If leftChild is greater than currentElement (i.e. element at i th index), set leftChildIndex as largest.
 - If rightChild is greater than element in largest, set rightChildIndex as largest.
6. Swap largest with currentElement.
7. Repeat steps 3-7 until the subtrees are also heapified.



Max Heap:

To create a Max-Heap:

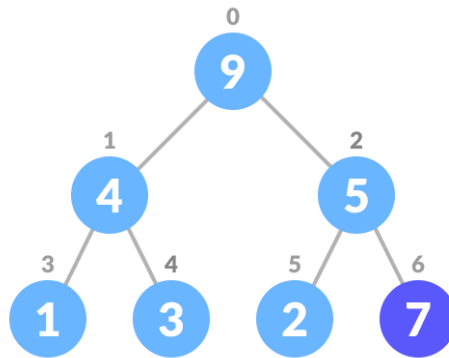
```
MaxHeap(array, size) {  
    loop from the first index of non-leaf node down to zero  
    call heapify  
}
```

For Min-Heap, both leftChild and rightChild must be larger than the parent for all nodes.

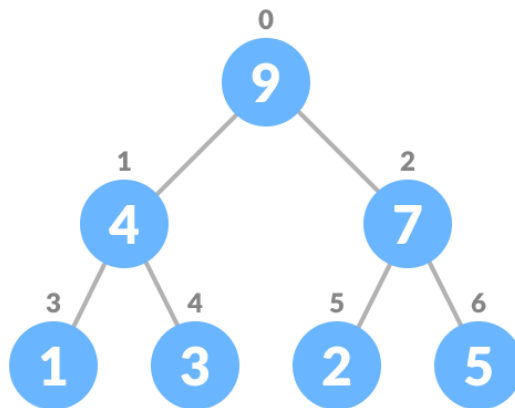
Insertion:

```
If there is no node,  
create a newNode.  
else (a node is already present)  
    insert the newNode at the end (last node from left to right.)  
heapify the array
```

1. Insert the new element at the end of the tree.



2. Heapify the tree.



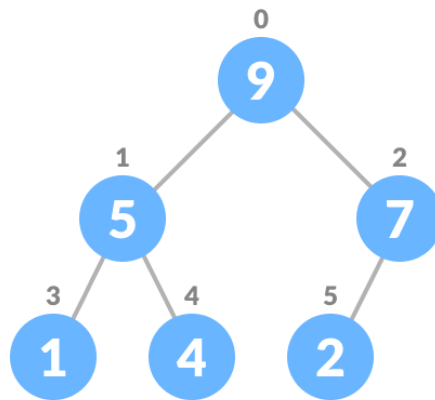
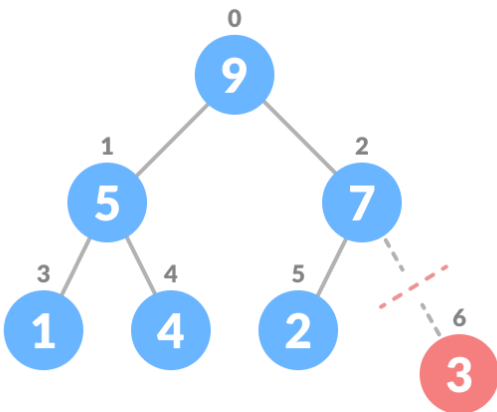
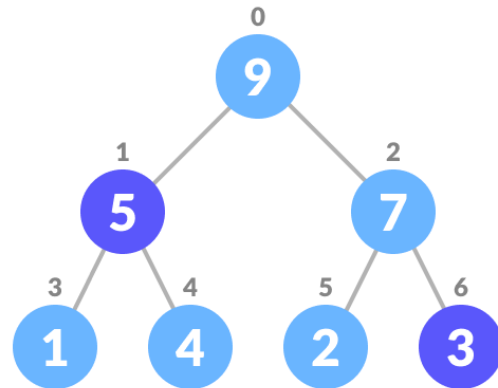
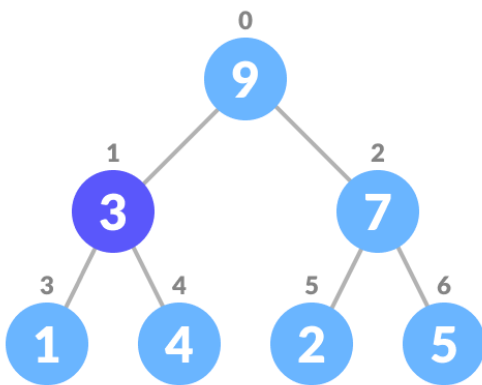
For Min Heap, the above algorithm is modified so that parentNode is always smaller than newNode.

Delete Element from Heap

Algorithm for deletion in Max Heap

```
If nodeToBeDeleted is the leafNode
    remove the node
Else swap nodeToBeDeleted with the lastLeafNode
    remove nodeToBeDeleted

heapify the array
```

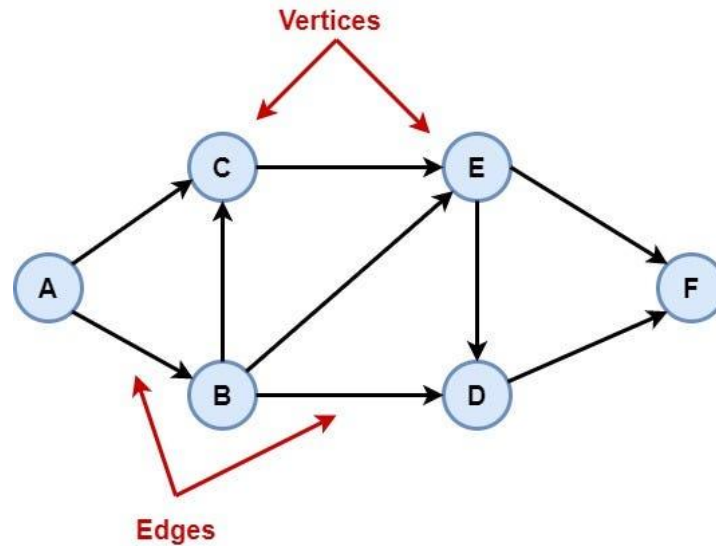


Heap Data Structure Applications

- Heap is used while implementing a priority queue.
- Dijkstra's Algorithm
- Heap Sort

Graph Data Structure

A Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph is composed of a set of vertices(V) and a set of edges(E). The graph is denoted by $G(V, E)$.



Representations of Graph

Here are the two most common ways to represent a graph: For simplicity, we are going to consider only **unweighted graphs** in this post.

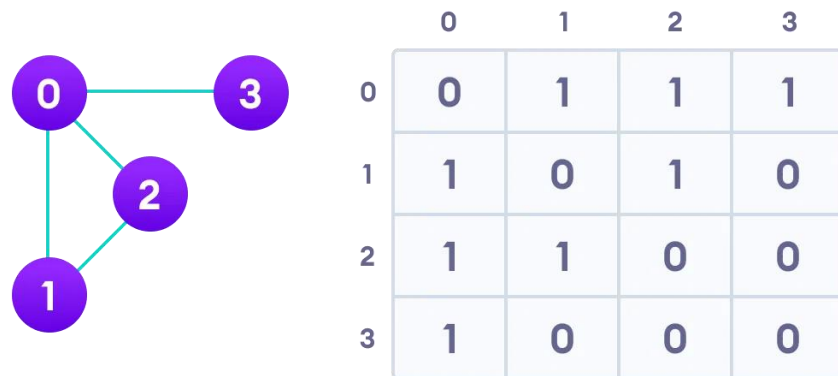
1. Adjacency Matrix
2. Adjacency List

Adjacency Matrix Representation

An adjacency matrix is a way of representing a graph as a boolean matrix of (0's and 1's) stored in 2D array of $V \times V$ vertices. Each row and column represent a vertex.

- If there is an edge from vertex i to j , mark $\text{adjMat}[i][j]$ as 1.
- If there is no edge from vertex i to j , mark $\text{adjMat}[i][j]$ as 0.

Representation of Undirected Graph as Adjacency Matrix:

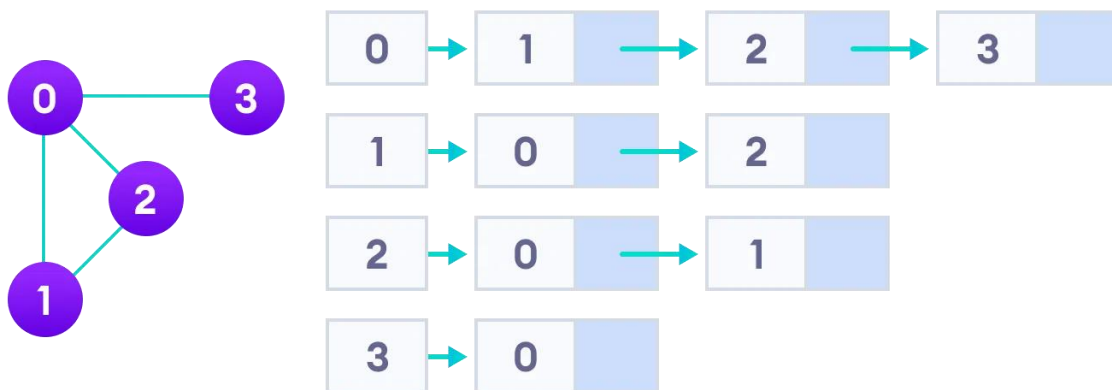


Adjacency List

An adjacency list represents a graph as an array of linked lists.

The index of the array represents a vertex and each element in its linked list represents the other vertices that form an edge with the vertex.

Representation of Undirected Graph as Adjacency List:



An adjacency list is efficient in terms of storage because we only need to store the values for the edges. For a graph with millions of vertices, this can mean a lot of saved space.

Graph Operations

The most common graph operations are:

- Check if the element is present in the graph
- Graph Traversal
- Add elements(vertex, edges) to graph
- Finding the path from one vertex to another